

Ciências da Computação – 6º Semestre Noturno (NACCOMP6)

Entrega 01 – Teoria da Computação e Linguagens Formais

Sweet Spot

Arthur Paltrinieri (24026559)

Beatriz de Souza Santos Rio Branco (23025365)

Fabiano Henrique Chou (24025991)

Marcelo Henrique de Souza (23025665)

Resumo do projeto

O projeto "Sweet Spot" consiste numa solução tecnológica para o treino solo de precisão no tênis, integrando visão computacional, IoT e um sistema web gamificado. O sistema utiliza uma maquete com uma parede de LEDs controlada por Arduino, onde os LEDs atuam como alvos. Duas câmeras processam a imagem em tempo real através de um modelo YOLOv11 treinado (Label Studio), detectando o impacto de bolas de isopor por meio de heurísticas (inversão de trajetória, escala ou perda de rastreamento). Os dados de precisão e tempo de reação são enviados para um backend Node.js/MySQL, que alimenta um *dashboard* interativo para análise de desempenho do jogador.

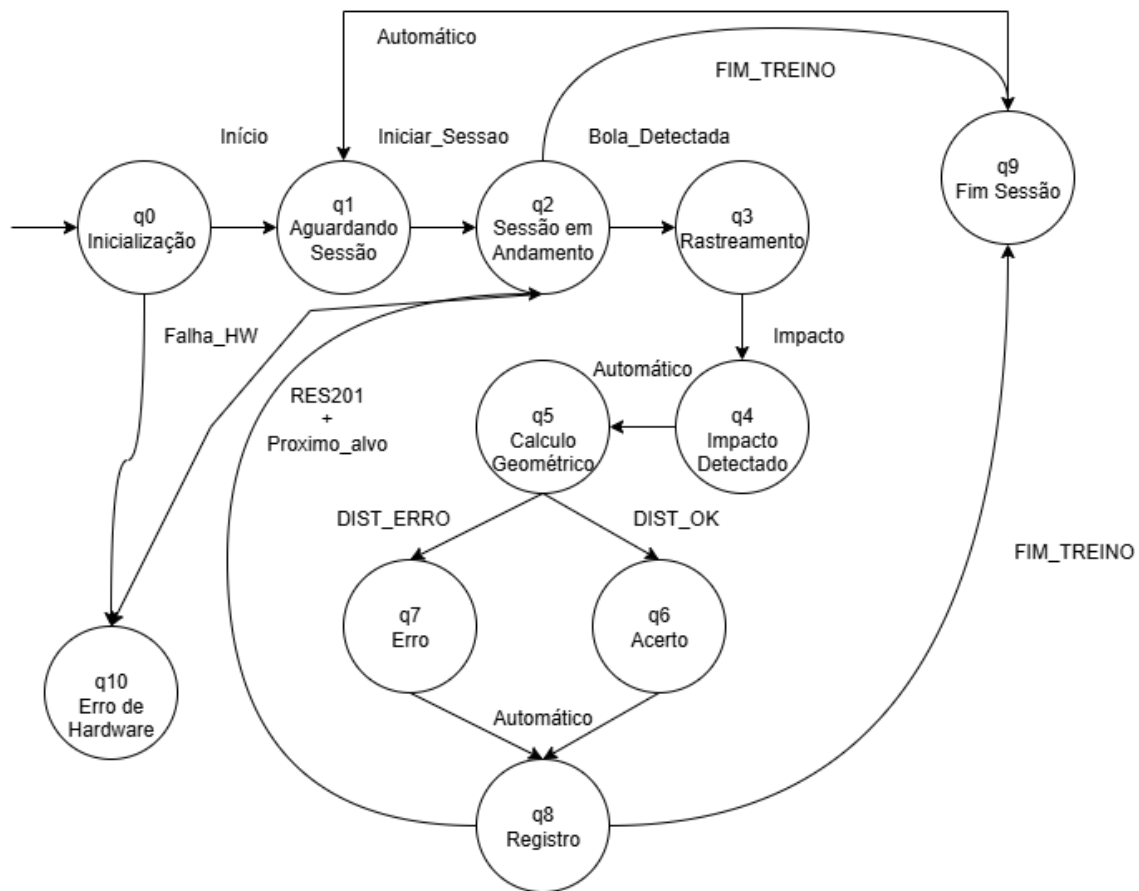
Identificação dos estados do sistema

Estado	Descrição
Q0 – Inicialização	O sistema inicia os scripts de visão computacional, carrega o modelo YOLO, estabelece conexão Serial com o Arduino e aguarda a calibração da câmara.
Q1 – Aguardando Sessão	O hardware e o software estão prontos. O sistema aguarda o comando do frontend (via API) para iniciar um novo treino.
Q2 – Sessão em Andamento	O treino inicia. O Arduino acende um LED aleatório (alvo) e o sistema passa a monitorizar a área de treino.
Q3 – Rastreamento (Tracking)	O modelo YOLO está ativamente a rastrear a bola de isopor no <i>frame</i> da câmara em tempo real.
Q4 – Impacto Detectado	Uma heurística de colisão (inversão de trajetória, inversão de escala ou <i>motion blur</i>) identifica que a bola atingiu a parede.
Q5 – Cálculo Geométrico	. O script calcula a distância em centímetros entre a coordenada (X,Y) do impacto detectado e o centro do LED ativo.
Q6 – Acerto (HIT)	A distância calculada é menor ou igual à tolerância definida (ex: 10 cm).
Q7 – Erro (MISS)	A distância calculada é superior à tolerância definida, ou a bola atingiu a parede fora da zona ativa.
Q8 – Registro (Telemetria)	Os dados da jogada (acerto/erro, precisão em %, tempo de reação) são enviados via requisição HTTP (POST) para a API Node.js.
Q9 – Fim de Sessão	O tempo do treino esgotou ou o limite de batidas foi atingido. Os dados finais são consolidados na base de dados MySQL.
Q10 – Erro de Hardware	Perda de conexão com as câmaras ou com a porta Serial do Arduino.

Eventos (entradas)

Evento	Símbolo	Descrição
Configuração Concluída	Início	Calibração e conexão com hardware concluídas.
Comando de Início	Iniciar_Sessao	O usuário clica em "Nova Sessão" no <i>dashboard</i> web.
Bola no Frame	Bola_Detectada	O YOLO identifica o objeto "bola" na imagem.
Gatilho de Colisão	Impacto	Mudança abrupta nos vetores de movimento ou perda de rastreo na zona calibrada.
Distância $\leq$ Tolerância	Dist_OK	Impacto próximo o suficiente do LED aceso.
Confirmação de API	RES201	O backend confirma a gravação da batida no banco de dados.
Novo Alvo Gerado	Proximo_Alvo	O Arduino recebe comando para acender outro LED.
Condição de Parada	Fim_Treino	O limite de batidas (ex: 10) ou o tempo da sessão foi atingido.
Desconexão	Falha_HW	Cabo USB desligado ou falha de leitura de vídeo/serial.

Diagrama de estados



O funcionamento inicia no estado **Q0 – Inicialização**, onde ocorre o mapeamento espacial (homografia) e a conexão Serial. Com sucesso, passa para **Q1 – Aguardando Sessão**. Ao receber o comando do utilizador, transita para **Q2 – Sessão em Andamento**, ativando os alvos.

A partir de **Q2**, sempre que a bola entra na área, transita para **Q3 – Rastreamento**. Quando as heurísticas do YOLO disparam, avança para **Q4 – Impacto Detectado**, seguido imediatamente de **Q5 – Cálculo Geométrico**.

Dependendo da distância medida em **Q5**, o sistema bifurca para **Q6 – Acerto** ou **Q7 – Erro**. Em ambos os casos, a transição seguinte é para **Q8 – Registro**, enviando os dados para a nuvem. Se a sessão ainda não terminou, o evento Proximo_Alvo devolve o sistema para **Q2**. Se o limite for atingido, o evento Fim_Treino leva a **Q9 – Fim de Sessão**, que ao finalizar o processamento regressa a **Q1**. Qualquer quebra física a meio leva ao estado de falha **Q10**.

Tabela de transições

Estado Atual	Evento (Entrada)	Próximo Estado
Q0 (Inicialização)	Inicio	Q1 (Aguardando Sessão)
Q0 (Inicialização)	Falha_HW	Q10 (Erro de Hardware)
Q1 (Aguardando Sessão)	Iniciar_Sessao	Q2 (Sessão em Andamento)
Q2 (Sessão em Andamento)	Bola_Detectada	Q3 (Rastreamento)
Q2 (Sessão em Andamento)	Fim_Treino	Q9 (Fim de Sessão)
Q2 (Sessão em Andamento)	Impacto	Q4 (Impacto Detectado)
Q4 (Impacto Detectado)	(Automático)	Q5 (Cálculo Geométrico)
Q5 (Cálculo Geométrico)	Dist_OK	Q6 (Acerto)
Q5 (Cálculo Geométrico)	Dist_Erro	Q7 (Erro)
Q6 (Acerto)	(Automático)	Q8 (Registro)
Q7 (Erro)	(Automático)	Q8 (Registro)
Q8 (Registro)	RES201 + Proximo_Alvo	Q2 (Sessão em Andamento)
Q8 (Registro)	RES201 + Fim_Treino	Q9 (Fim de Sessão)
Q9 (Fim de Sessão)	(Automático)	Q1 (Aguardando Sessão)
Qualquer Estado	Falha_HW	Q10 (Erro de Hardware)

Linguagem do sistema

Para a linguagem formal, é possível definir o conjunto de eventos utilizados pelo sistema como:

$\Sigma = \{\text{INICIO, INICIAR_SESSAO, BOLA_DETECTADA, IMPACTO, DIST_OK, DIST_ERRO, RES201, PROXIMO_ALVO, FIM_TREINO, FALHA_HW}\}$

Sequência válida 1 – Treino com Acerto Único: INICIO → INICIAR_SESSAO → BOLA_DETECTADA → IMPACTO → DIST_OK → RES201 → FIM_TREINO

Sequência válida 2 – Treino com Dois Remates (Acerto e Erro): INICIO → INICIAR_SESSAO → BOLA_DETECTADA → IMPACTO → DIST_OK → RES201 → PROXIMO_ALVO → BOLA_DETECTADA → IMPACTO → DIST_ERRO → RES201 → FIM_TREINO

Expressão Regular

A expressão regular abaixo descreve o fluxo de uma sessão completa, aceitando múltiplos ciclos de jogadas (acertos ou erros) até ao fim da sessão:

INICIO INICIAR_SESSAO (BOLA_DETECTADA IMPACTO (DIST_OK | DIST_ERRO) RES201 PROXIMO_ALVO)*FIM_TREINO

Gramática

Definição formal $G = (V, \Sigma, P, S)$:

Não terminais (V):

- S : Sequência do Sistema
- C : Ciclo de Sessão
- J : Jogada (Tracking e Impacto)
- R : Resultado da Batida

Produções (P):

- $S \rightarrow \text{INICIO INICIAR_SESSAO } C$
- $C \rightarrow J \text{ RES201 PROXIMO_ALVO } C$
- $C \rightarrow J \text{ RES201 FIM_TREINO}$
- $J \rightarrow \text{BOLA_DETECTADA IMPACTO } R$
- $R \rightarrow \text{DIST_OK}$
- $R \rightarrow \text{DIST_ERRO}$

Justificativa da modelagem

A modelagem baseada em máquinas de estados finitos (MEF) é ideal para o Sweet Spot, visto que o sistema opera através de um *loop* síncrono bem definido entre hardware e software. As operações de visão computacional dependem rigorosamente de estados anteriores: o modelo YOLO (estado de rastreamento) só tem utilidade contextual se uma sessão tiver sido iniciada e um LED alvo estiver ativo.

A separação dos estados *Q4* (Impacto), *Q5* (Cálculo) e bifurcação para *Q6/Q7* (Acerto/Erro) ilustra de forma explícita a arquitetura lógica do projeto: primeiro a rede neural identifica a colisão temporal e espacialmente, e apenas depois a matemática clássica (distância euclidiana baseada na homografia) julga a qualidade da ação.

Além disso, a introdução do estado *Q8* (Registro) reflete a natureza IoT do projeto, evidenciando que o ciclo de uma jogada só se conclui quando a integração com a API RESTful e a base de dados MySQL tem sucesso, permitindo que a gramática regular represente perfeitamente o ciclo de *gamification* do sistema.